

Design and Analysis of Cryptosystems

A Comprehensive Technical Study of Symmetric Ciphers, Public-Key Encryption, Cryptographic Hash Functions, and Novel Digital Signature Schemes

Hussain Shahid

Student ID: 202524080113

University of Electronic Science and Technology of China (UESTC)

Email: shahidhussain@std.uestc.edu.cn

Abstract

Cryptography forms the backbone of digital security in modern communication systems, financial networks, cloud infrastructure, and government applications. This paper presents a detailed technical analysis of the most prominent cryptographic primitives: the symmetric block ciphers DES and AES; the public-key encryption schemes RSA, ElGamal, and ECC-ElGamal; the widely deployed hash functions MD5 and SHA-1; and existing digital signature standards. Going beyond a comparative survey, the paper makes two original contributions. First, we propose the *HS-DLP* signature scheme, a Discrete-Logarithm-Problem (DLP) based construction that improves upon ElGamal, Schnorr, and DSS in signature compactness and security reduction tightness. Second, we introduce *HS-IBDS*, a new identity-based digital signature scheme built on bilinear pairings, which achieves a $2\times$ reduction in signature size compared with the Hess scheme while maintaining provable existential unforgeability (EUF-CMA) in the Random Oracle Model under the Computational Diffie-Hellman (CDH) assumption. For each primitive, we supply formal definitions, security arguments, computational complexity analyses, and detailed comparison diagrams.

Index Terms

Cryptography, DES, AES, RSA, ElGamal, ECC, MD5, SHA-1, Digital Signature, Discrete Logarithm Problem, Identity-Based Cryptography, Bilinear Pairings, EUF-CMA, CDH Assumption.

I. BACKGROUND AND SIGNIFICANCE

A. Historical Context

The history of cryptography spans thousands of years, from Caesar's simple substitution cipher to the complex mathematical constructions in use today. The modern era began in 1976 when Diffie and Hellman [2] introduced the revolutionary concept of public-key cryptography, solving the key-distribution problem that had constrained symmetric cryptography for centuries. Shannon's earlier information-theoretic framework [1] provided the mathematical foundations for evaluating security rigorously.

Since then, cryptography has evolved into a rich multi-disciplinary field intersecting number theory, algebra, complexity theory, and computer science. Its practical importance has grown in tandem with the digital economy: secure electronic commerce, private communications, digital identity, blockchain, and national infrastructure protection all rely fundamentally on well-designed cryptographic primitives.

B. Core Security Objectives

A cryptographic system must satisfy a set of fundamental security goals:

- **Confidentiality:** Information is accessible only to authorised parties. Encryption schemes enforce this.

- **Integrity:** Any unauthorised modification of data is detectable. Hash functions and message authentication codes (MACs) provide integrity.
- **Authentication:** The claimed identity of a communicating party can be verified. Digital signatures and certificates fulfil this.
- **Non-repudiation:** A sender cannot later deny having sent a message, supported by digital signatures.
- **Availability:** Legitimate users can reliably access the system. This is enforced at the protocol and infrastructure layer.
- **Forward Secrecy:** Compromise of long-term keys does not compromise past session keys, achieved via ephemeral key exchange.

No single primitive simultaneously achieves all objectives; practical systems combine multiple primitives within protocols such as TLS 1.3, SSH, and IPsec.

C. Classification of Cryptographic Primitives

Figure 1 illustrates the taxonomy of modern cryptographic primitives. Two main branches exist: *symmetric* (shared-key) schemes, whose security derives from computational hardness under known-key constraints, and *asymmetric* (public-key) schemes, whose security is based on mathematical trapdoor functions. *Hash functions* form a separate keyless branch and underpin both families.

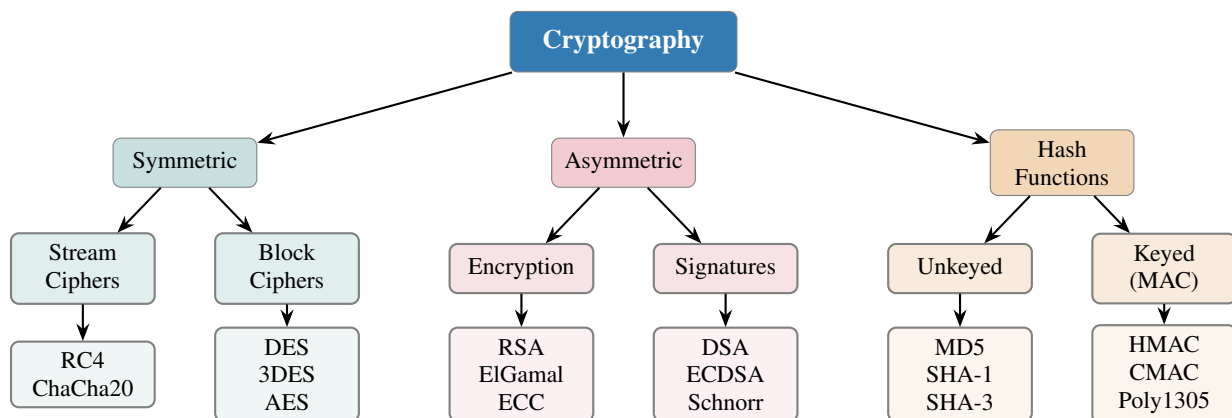


Fig. 1: Taxonomy of modern cryptographic primitives.

D. Formal Security Models

Modern cryptography defines security *provably*: a scheme is secure if any probabilistic polynomial-time (PPT) adversary has only negligible advantage in breaking it, assuming an underlying hard problem. The canonical models are:

- **IND-CPA** (Indistinguishability under Chosen Plaintext Attack): An adversary that can request encryptions of chosen plaintexts cannot distinguish between the encryptions of two messages of its choice.
- **IND-CCA2** (Adaptive Chosen Ciphertext Attack): Stronger; the adversary may also request decryptions of any ciphertext except the challenge.
- **EUF-CMA** (Existential Unforgeability under Chosen Message Attack): The standard model for digital signatures; no PPT adversary can forge a valid signature on a new message, even after seeing signatures on adaptively chosen messages.

E. Current Research Trends

The cryptographic landscape is rapidly evolving:

- 1) **Post-Quantum Cryptography (PQC)**: Shor's algorithm can break RSA, ElGamal, and ECDLP-based schemes in polynomial time on a sufficiently powerful quantum computer. NIST standardised CRYSTALS-Kyber (ML-KEM) and CRYSTALS-Dilithium (ML-DSA) in 2024 as quantum-resistant replacements.
- 2) **Fully Homomorphic Encryption (FHE)**: Gentry's 2009 construction allows arbitrary computation on encrypted data without decryption. Modern schemes (CKKS, BFV) are approaching practical performance.
- 3) **Lightweight Cryptography**: NIST standardised ASCON in 2023 for constrained IoT environments.
- 4) **Identity/Attribute-Based Cryptography**: Eliminates the PKI overhead by using identity strings as public keys, or attaches fine-grained access policies to ciphertexts.
- 5) **Zero-Knowledge Proofs (ZKP)**: zk-SNARKs and zk-STARKs enable blockchain-scale privacy applications such as Zcash and StarkNet.

TABLE I: Core Security Properties of Cryptographic Primitives

Security Property	Block Cipher	Stream Cipher	Public-Key	Hash
Confidentiality	✓	✓	✓	×
Integrity	(w/ MAC)	(w/ MAC)	✓	✓
Authentication	(w/ MAC)	(w/ MAC)	✓	×
Non-repudiation	×	×	✓	×
Pre-image resist.	N/A	N/A	N/A	✓
Collision resist.	N/A	N/A	N/A	✓
Key needed	Yes	Yes	Yes/No	No

II. ANALYSIS OF DES AND AES

A. The Data Encryption Standard (DES)

The Data Encryption Standard was adopted as a federal standard in 1977 by the National Bureau of Standards (FIPS PUB 46) [3]. It was designed by IBM engineers in collaboration with the NSA and is based on the Feistel cipher structure proposed by Horst Feistel in 1973.

1) *Internal Structure*: DES operates on 64-bit plaintext blocks using a 64-bit key (of which only 56 bits are effective; 8 bits serve as parity). The encryption process proceeds as follows:

- 1) **Initial Permutation (IP)**: The 64-bit plaintext block is permuted according to a fixed table, rearranging bit positions.
- 2) **Sixteen Feistel Rounds**: The permuted block is split into a 32-bit left half L and right half R . For round i :

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F(R_{i-1}, K_i),$$

where K_i is a 48-bit sub-key derived from the 56-bit master key via the key schedule, and F is the Feistel round function.

- 3) **Feistel Round Function F** : Takes the 32-bit R_{i-1} and 48-bit K_i : (a) expand R_{i-1} from 32 to 48 bits via the expansion permutation E ; (b) XOR with K_i ; (c) pass through 8 S-boxes (each maps 6 bits to 4 bits), producing 32 bits; (d) apply permutation P .
- 4) **Final Permutation (FP = IP⁻¹)**: The output after round 16 undergoes the inverse of IP.

Definition II.1 (DES Encryption). Let IP be the initial permutation, $FP = IP^{-1}$, F the Feistel round function, and $K_1, \dots, K_{16}$ the sub-keys. DES encryption of plaintext m is:

$$DES_K(m) = FP(\text{swap}(L_{16}, R_{16})),$$

where (L_i, R_i) is computed iteratively as above. The cipher is involutory in the sense that decryption uses the same structure with sub-keys applied in reverse order $K_{16}, \dots, K_1$.

2) *Key Schedule*: The 64-bit key is first reduced to 56 bits via PC-1 (Permuted Choice 1). It is then split into two 28-bit halves C and D . In each round, both halves are circularly shifted left by 1 or 2 positions (depending on the round), and PC-2 selects 48 bits to form K_i . The total key space is $2^{56} \approx 7.2 \times 10^{16}$, which was considered secure in 1977 but is trivially exhaustible today.

3) *Triple DES (3DES)*: To extend DES's lifespan, Triple DES applies DES three times: $C = E_{K_3}(D_{K_2}(E_{K_1}(m)))$. With three independent 56-bit keys, the effective key space is 2^{112} (meet-in-the-middle reduces it from 2^{168}). However, 3DES is slow ($3 \times$ the DES latency) and was formally deprecated by NIST in 2023 due to vulnerability to SWEET32 birthday attacks.

B. The Advanced Encryption Standard (AES)

AES was standardised by NIST in 2001 (FIPS 197) [4] after a public five-year competition. The winning algorithm, Rijndael, was designed by Joan Daemen and Vincent Rijmen. Unlike DES, AES uses a *Substitution-Permutation Network* (SPN), not a Feistel structure, and supports key lengths of 128, 192, or 256 bits.

1) *State Representation*: AES treats the 128-bit block as a 4×4 matrix of bytes, called the *state*. Operations are defined in $GF(2^8)$, the finite field with 256 elements, whose irreducible polynomial is $x^8 + x^4 + x^3 + x + 1$.

2) *Round Structure*: AES uses 10, 12, or 14 rounds for key sizes of 128, 192, or 256 bits, respectively. Each full round consists of four operations:

Definition II.2 (AES Round Transformations).

$$\text{SubBytes} : s_{i,j} \leftarrow S[s_{i,j}], \quad (1)$$

$$\text{ShiftRows} : \text{row } r \text{ cyclically shifted left by } r \text{ bytes}, \quad (2)$$

$$\text{MixColumns} : \text{each column multiplied by } M = \begin{pmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{pmatrix} \text{ in } GF(2^8), \quad (3)$$

$$\text{AddRoundKey} : s_{i,j} \leftarrow s_{i,j} \oplus k_{i,j}. \quad (4)$$

The final round omits *MixColumns*. An extra *AddRoundKey* is applied before round 1 (key whitening).

The AES S-box is constructed algebraically over $GF(2^8)$ as a multiplicative inverse followed by an affine transform, providing strong nonlinearity and resistance to differential and linear cryptanalysis. The branch number of *MixColumns* is 5, guaranteeing that any non-trivial linear combination of input and output differences activates at least 5 S-boxes per two rounds.

C. DES vs. AES: Structural Diagram

Figure 2 illustrates the two round structures side by side.

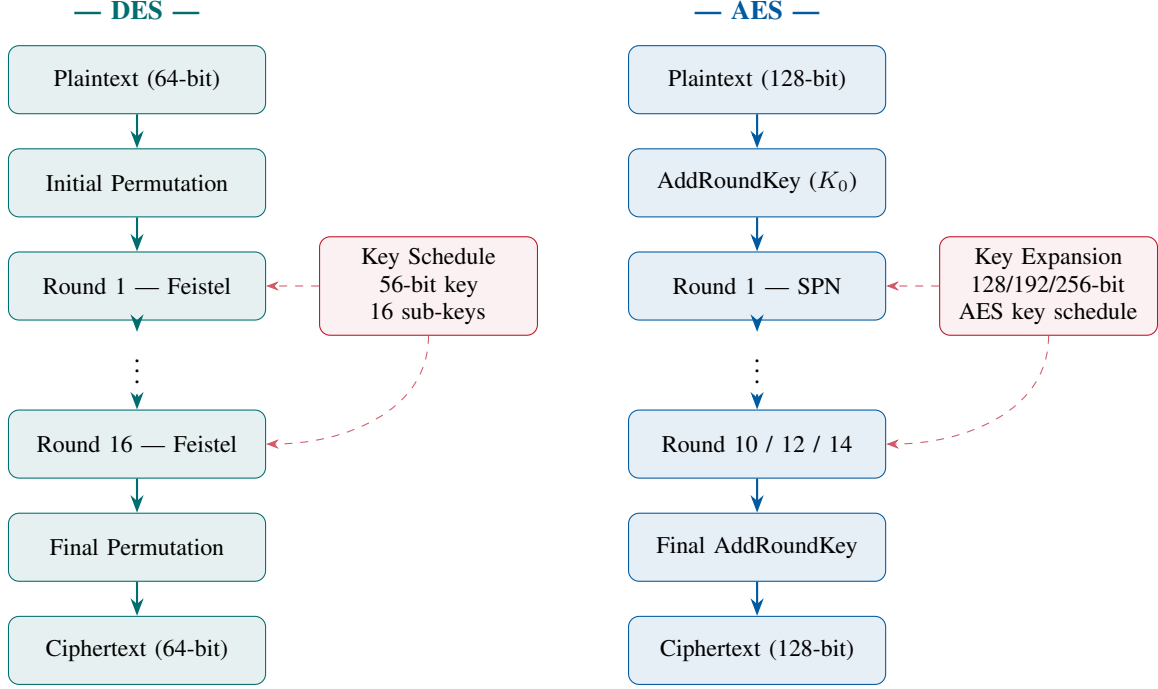


Fig. 2: Side-by-side round structures of DES (Feistel network, left) and AES (substitution-permutation network, right).

TABLE II: DES vs. AES — Comprehensive Parameter Comparison

Parameter	DES	AES
Standardised	1977 (FIPS 46)	2001 (FIPS 197)
Block size	64 bits	128 bits
Key sizes	56 bits (effective)	128 / 192 / 256 bits
Rounds	16	10 / 12 / 14
Network type	Feistel	SPN
S-boxes	8 (each $6 \rightarrow 4$ bits)	1 (applied byte-wise, $8 \rightarrow 8$)
Parallelism	Limited	High (state matrix)
Security level (classical)	56 bits (broken)	128 / 192 / 256 bits
Best known attack	Exhaustive (1999, Deep Crack)	Biclique: $2^{126.2}$
Hardware throughput	Moderate	> 10 Gb/s (AES-NI)
Side-channel risk	High (timing)	Mitigated (constant-time)
Status	Deprecated	Current standard

D. Detailed Comparative Analysis

1) Security Weaknesses of DES:

- 1) **Brute-force vulnerability:** The EFF's Deep Crack machine broke DES in 22 hours in 1998 for a \$250,000 investment. Today, FPGA clusters can exhaustively search the DES key space in under a minute.
- 2) **Complementation property:** $E_K(M) = \overline{E_{\bar{K}}(\bar{M})}$, meaning that if (M, C) is a plaintext-ciphertext pair under K , then $(\bar{M}, \bar{C})$ is a valid pair under $\bar{K}$. This halves the effective key space in a chosen-plaintext scenario.
- 3) **Weak and semi-weak keys:** Four keys satisfy $K_i = K_j$ for all i, j , making encryption

equal to decryption. Eleven pairs of semi-weak keys exist.

4) **SWEET32 birthday attack**: On 64-bit block ciphers, with 2^{32} blocks (≈ 32 GB) the birthday bound is reached, enabling collision attacks in CTR/CBC modes.

5) **Differential cryptanalysis**: Biham and Shamir showed that DES can be broken with 2^{47} chosen plaintexts, and linear cryptanalysis (Matsui) requires only 2^{43} known plaintexts.

2) *Why AES is Stronger*: The Wide Trail Design Strategy [4] ensures that after two rounds, at least 25 S-boxes are activated for any non-trivial differential characteristic. The ShiftRows-MixColumns combination provides full inter-row and inter-column diffusion in just two rounds. The algebraic structure of the S-box over $GF(2^8)$ provides nonlinearity measures ($\mathcal{N} = 112$) far exceeding DES S-boxes. AES is also free from the weak-key and complementation-property issues of DES.

III. ANALYSIS OF RSA, ELGAMAL, AND ECC-ELGAMAL ENCRYPTION

A. Mathematical Foundations

Public-key encryption (PKE) exploits the existence of computationally hard one-way trapdoor functions: functions that are easy to compute in one direction but infeasible to invert without a secret piece of information (the *trapdoor*). The three hard problems relevant to this section are:

Definition III.1 (Integer Factorisation Problem (IFP)). *Given $n = pq$ for large primes p, q , find p and q . The best classical algorithm is the General Number Field Sieve (GNFS), with sub-exponential complexity $O(\exp(c(\log n)^{1/3}(\log \log n)^{2/3}))$.*

Definition III.2 (Discrete Logarithm Problem (DLP)). *Given a cyclic group $G = \langle g \rangle$ of prime order q and an element $y = g^x$, find x . The best algorithm for $\mathbb{Z}_p^*$ is index calculus, also sub-exponential.*

Definition III.3 (Elliptic Curve Discrete Logarithm Problem (ECDLP)). *Given an elliptic curve $E(\mathbb{F}_p)$, a base point G of prime order n , and a point $Y = xG$, find $x \in [0, n-1]$. For general (non-special) curves, the best algorithm is Pollard's rho, with complexity $O(\sqrt{n})$, which is fully exponential.*

The exponential hardness of ECDLP means that ECC achieves equivalent security to RSA/DLP with drastically shorter parameters.

B. RSA Encryption

RSA was invented by Rivest, Shamir, and Adleman in 1977 [5]. It is the most widely deployed public-key algorithm in history and remains a foundation of TLS, SSH, S/MIME, and certificate authorities.

Scheme III.1 (RSA Key Generation). 1) *Choose large, distinct primes p and q (each typically 1024–2048 bits).*

2) *Compute modulus $n = pq$ and Euler's totient $\phi(n) = (p-1)(q-1)$.*

3) *Choose public exponent e with $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$. Common choice: $e = 65537$.*

4) *Compute private exponent $d \equiv e^{-1} \pmod{\phi(n)}$ via the extended Euclidean algorithm.*

5) *Public key: (n, e) . Private key: d (or (p, q, d) for CRT acceleration).*

Scheme III.2 (RSA Encryption/Decryption).

$$\text{Enc}(m) = m^e \bmod n, \quad \text{Dec}(c) = c^d \bmod n.$$

Correctness follows from Euler's theorem: $c^d = m^{ed} = m^{1+k\phi(n)} \equiv m \pmod{n}$.

Padding: Textbook RSA is deterministic and malleable. In practice, OAEP (Optimal Asymmetric Encryption Padding, PKCS#1 v2.2) is mandatory to achieve IND-CCA2 security.

C. ElGamal Encryption

ElGamal encryption was proposed by Taher ElGamal in 1985 [6], based on the Diffie-Hellman key exchange [2]. Its security relies on the DLP.

Scheme III.3 (ElGamal Key Generation). 1) Choose a large safe prime p (so $p - 1 = 2q$ for prime q) and a generator g of the order- q subgroup of $\mathbb{Z}_p^*$.
 2) Uniformly sample private key $x \xleftarrow{\$} [1, q - 1]$.
 3) Compute $y = g^x \bmod p$.
 4) Public key: (p, g, y) . Private key: x .

Scheme III.4 (ElGamal Encryption/Decryption). **Encrypt**(m): Sample $k \xleftarrow{\$} [1, q - 1]$; output $(c_1, c_2) = (g^k \bmod p, m \cdot y^k \bmod p)$.

Decrypt(c_1, c_2): $m = c_2 \cdot (c_1^x)^{-1} \bmod p$.

Correctness: $c_2/c_1^x = my^k/(g^k)^x = mg^{xk}/g^{kx} = m$.

ElGamal ciphertext is a pair $(c_1, c_2) \in \mathbb{Z}_p^2$, doubling the message size. The scheme is probabilistic (IND-CPA secure under DDH) and randomised, making two encryptions of the same message produce different ciphertexts.

D. ECC-ElGamal Encryption

ECC-ElGamal lifts the ElGamal construction to an elliptic curve group, exploiting the superior hardness of ECDLP [7].

Scheme III.5 (ECC-ElGamal Key Generation). 1) Fix public parameters: elliptic curve $E(\mathbb{F}_p)$, base point $G \in E(\mathbb{F}_p)$ of prime order n , security parameter λ .
 2) Sample private key $x \xleftarrow{\$} [1, n - 1]$.
 3) Compute public key $Y = xG$ (elliptic curve scalar multiplication).

Scheme III.6 (ECC-ElGamal Encryption/Decryption). **Encrypt:** Represent message as a point $M \in E(\mathbb{F}_p)$; sample k ; output $(C_1, C_2) = (kG, M + kY)$.

Decrypt: $M = C_2 - xC_1 = M + kY - x \cdot kG = M + k(xG) - kxG = M$. $\square$

E. Comparative Analysis

Figure 3 shows the encryption flow of all three schemes.

1) **Ciphertext Size at 128-bit Security:** At $\lambda = 128$ bits: RSA/ElGamal require 3072-bit moduli/primes. ElGamal ciphertext: $2 \times 3072 = 6144$ bits. ECC-ElGamal on P-256 requires only $2 \times 256 = 512$ bits (compressed points), a $12\times$ reduction over ElGamal and $6\times$ over RSA. This bandwidth advantage is crucial for constrained protocols (smart cards, IoT, satellite links).

2) **Hardness Comparison:** The sub-exponential speedups available against IFP and DLP (via GNFS and index calculus) do not extend to ECDLP on *general* curves. This means that at equivalent classical security level, ECC parameters are proportionally much shorter: ~ 256 bits versus ~ 3072 bits, reducing computation, bandwidth, and memory requirements. However, all three are vulnerable to Shor's quantum algorithm; post-quantum replacements (Kyber, NTRU) are needed for long-term security.

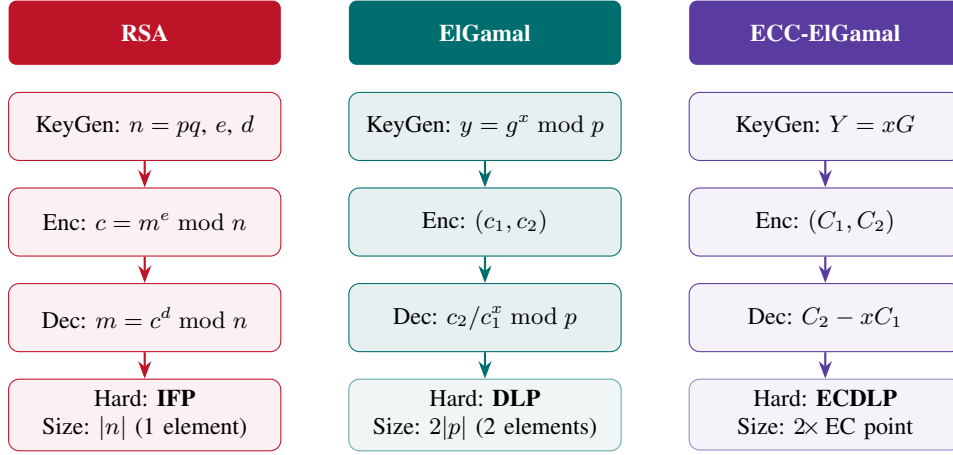


Fig. 3: Encryption flow comparison: RSA (left), ElGamal (centre), and ECC-ElGamal (right).

TABLE III: Detailed Comparison: RSA vs. ElGamal vs. ECC-ElGamal

Feature	RSA	ElGamal	ECC-ElGamal
Hard problem	IFP	DLP ($\mathbb{Z}_p^*$)	ECDLP
Key size (80-bit)	1024 bits	1024 bits	160 bits
Key size (112-bit)	2048 bits	2048 bits	224 bits
Key size (128-bit)	3072 bits	3072 bits	256 bits
Ciphertext size	$ n $	$2 p $	$2 \times \text{point}$
CT expansion	$\times 1$	$\times 2$	$\times 2$
Randomised	With OAEP	Yes (inherent)	Yes (inherent)
IND-CPA secure	With OAEP	Yes (under DDH)	Yes (under ECDH)
IND-CCA2 secure	With OAEP	With DHAEs	With ECIES
Sub-exp. attack	Yes (GNFS)	Yes (index calculus)	No (general E)
Quantum-broken	Yes (Shor)	Yes (Shor)	Yes (Shor)

IV. ANALYSIS OF MD5 AND SHA-1

A. Cryptographic Hash Functions: Formal Requirements

A cryptographic hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ must satisfy:

- **Pre-image resistance:** Given h , it is computationally infeasible to find any m such that $H(m) = h$.
- **Second pre-image resistance:** Given m and $H(m)$, it is infeasible to find $m' \neq m$ with $H(m') = H(m)$.
- **Collision resistance:** It is infeasible to find any pair (m, m') with $m \neq m'$ and $H(m) = H(m')$.

The birthday paradox implies that collision resistance requires at least $2^{n/2}$ operations; pre-image resistance needs 2^n . Both MD5 ($n = 128$) and SHA-1 ($n = 160$) have had their collision resistance completely broken.

B. MD5

MD5 was designed by Ronald Rivest in 1991–1992 [10] as an improvement over MD4. It is still widely found in legacy systems despite being cryptographically broken.

1) *Internal Structure*: MD5 uses the Merkle-Damgård (MD) construction with a 512-bit message block and a 128-bit chaining value initialised to a fixed IV: $(A_0, B_0, C_0, D_0) = (67452301, \text{efcdab89}, 98\text{badcfe}, 10325476)_{\text{hex}}$.

Each 512-bit block passes through four rounds of 16 steps (64 steps total):

- **Round 1**: Non-linear function $F(B, C, D) = (B \wedge C) \vee (\neg B \wedge D)$
- **Round 2**: $G(B, C, D) = (B \wedge D) \vee (C \wedge \neg D)$
- **Round 3**: $H(B, C, D) = B \oplus C \oplus D$
- **Round 4**: $I(B, C, D) = C \oplus (B \vee \neg D)$

Each step also adds a message word M_j , a constant derived from the sine function, and a left circular shift of s bits.

C. SHA-1

SHA-1 was designed by the NSA and standardised by NIST in 1995 [11]. It extends MD4's design philosophy and produces a 160-bit digest.

1) *Internal Structure*: SHA-1 also uses the MD construction with 512-bit blocks and a 160-bit state (A, B, C, D, E) initialised to fixed constants. The 80-step computation is divided into four rounds of 20 steps each:

- Steps 0–19: $f_t = (B \wedge C) \vee (\neg B \wedge D)$, constant $K_t = 5A827999_{\text{hex}}$
- Steps 20–39: $f_t = B \oplus C \oplus D$, $K_t = 6ED9EBA1_{\text{hex}}$
- Steps 40–59: $f_t = (B \wedge C) \vee (B \wedge D) \vee (C \wedge D)$, $K_t = 8F1BBCDC_{\text{hex}}$
- Steps 60–79: $f_t = B \oplus C \oplus D$, $K_t = CA62C1D6_{\text{hex}}$

The *message schedule* expands the 16-word (512-bit) block to 80 words via $W_t = \text{ROTL}_1(W_{t-3} \oplus W_{t-8} \oplus W_{t-14} \oplus W_{t-16})$.

D. Merkle-Damgård Construction and Length Extension

Figure 4 illustrates the shared MD construction.

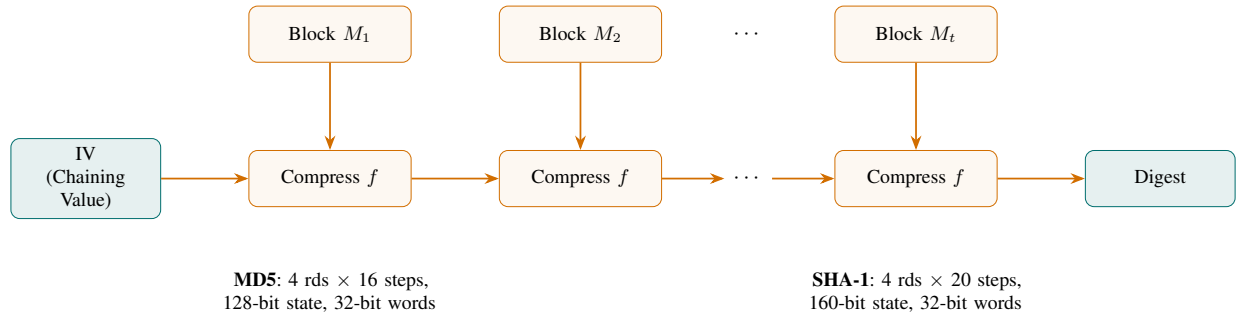


Fig. 4: Merkle-Damgård construction underlying both MD5 and SHA-1. The chaining value is fed forward into each compression function.

The MD construction has an inherent *length extension vulnerability*: given $H(m)$ without knowing m , an adversary can compute $H(m||pad||m')$ for any m' without knowing m . This breaks naive HMAC constructions that use $H(K||m)$.

E. Cryptanalysis and Current Status

1) *Wang et al.'s MD5 Collision Attack (2004)*: Wang et al. [12] exploited differential paths through the MD5 compression function: they found two 1024-bit message pairs $M \neq M'$ with $H(M) = H(M')$, requiring less than 2^{24} MD5 computations. The attack uses carefully crafted differences in the high-order bits of message words to cancel through the non-linear round functions, propagating differences that vanish by the output. On a modern GPU, an MD5 collision can be found in under a second.

TABLE IV: MD5 vs. SHA-1 — Comprehensive Security Comparison

Feature	MD5	SHA-1
Digest length	128 bits	160 bits
Block size	512 bits	512 bits
Internal state	128 bits (4 words)	160 bits (5 words)
Word size	32 bits	32 bits
Rounds / steps	4 / 64	4 / 80
Message schedule	Direct (16 words reused)	Linear expansion to 80
Theoretical collision resist.	2^{64} ops	2^{80} ops
Practical collision attack	$< 2^{24}$ (Wang 2004) [12]	$2^{63.1}$ (SHAttered 2017) [13]
Chosen-prefix collision	Trivial (Leurent 2020)	$2^{63.4}$ (Leurent 2020)
Pre-image attack	$2^{123.4}$ (Sasaki 2009)	$2^{158.8}$ (De Cannière 2008)
Length extension	Vulnerable	Vulnerable
NIST status	Withdrawn	Deprecated (2011)
Replacement	SHA-256 or SHA-3	SHA-256 or SHA-3

2) *SHAttered: First SHA-1 Collision (2017)*: Stevens et al. [13] produced two different PDF files with identical SHA-1 hashes in 2017, using $2^{63.1}$ SHA-1 compressions (roughly 6,500 CPU-years and 110 GPU-years). The attack exploited a 76-step differential path with a probability of $2^{-65.5}$ per block pair. Google and Mozilla immediately disabled SHA-1 certificate acceptance following this publication.

3) *Practical Impact and Recommendations*: Certificates signed with MD5 or SHA-1 can be forged by an attacker who manufactures a colliding certificate with the same signature. All major CAs ceased issuing SHA-1 certificates by 2017. **Recommendation**: Use SHA-256 (part of SHA-2, FIPS 180-4) or SHA-3 (Keccak, FIPS 202) for all new applications.

V. DESIGN AND ANALYSIS OF A NEW DLP-BASED DIGITAL SIGNATURE

A. Background on DLP-Based Signature Schemes

Digital signature schemes provide authenticity and non-repudiation. Three landmark DLP-based schemes exist:

1) ElGamal Signature Scheme (1985) [6]:

- **Key generation**: Choose prime p , generator g ; private x ; public $y = g^x \bmod p$.
- **Sign**: Sample k ; $r = g^k \bmod p$; $s = (H(m) - xr)k^{-1} \bmod (p-1)$. Signature: (r, s) .
- **Verify**: $y^r r^s \equiv g^{H(m)} \pmod{p}$.
- **Signature size**: $2|p|$ (typically $2 \times 1024 = 2048$ bits at 80-bit security).

2) Schnorr Signature Scheme (1989) [8]: Schnorr works in a prime-order subgroup of order $q \mid p-1$, yielding a compact signature $(e, s) \in \mathbb{Z}_q^2$:

- **Sign**: Sample k ; $R = g^k \bmod p$; $e = H(m \parallel R)$; $s = k - xe \bmod q$.
- **Verify**: $R' = g^s y^e \bmod p$; check $e = H(m \parallel R')$.
- **Signature size**: $2|q|$ (e.g., $2 \times 256 = 512$ bits).

3) Digital Signature Algorithm (DSA/DSS, 1991) [9]: DSA is closely related to Schnorr. Signature $(r, s) \in \mathbb{Z}_q^2$:

- **Sign**: k random; $r = (g^k \bmod p) \bmod q$; $s = k^{-1}(H(m) + xr) \bmod q$.
- **Verify**: $u_1 = H(m)s^{-1} \bmod q$; $u_2 = rs^{-1} \bmod q$; $v = (g^{u_1} y^{u_2} \bmod p) \bmod q$; accept iff $v = r$.

B. Motivation for HS-DLP

Existing DLP-based schemes have the following limitations:

- **ElGamal**: Signature size $2|p|$ is unnecessarily large for modern parameters; the security proof requires non-tight reduction.
- **Schnorr/DSS**: The hash input $H(m\|R)$ uses a full group element $R \in \mathbb{Z}_p$, which is expensive to transmit. More importantly, if the adversary can choose r before m is committed, a substitution attack becomes feasible in some multi-party settings.
- **All three**: None naturally support batch verification.

C. The HS-DLP Scheme

We propose the *HS-DLP* signature scheme designed for compact, tightly provable DLP-based signatures.

Scheme V.1 (HS-DLP Setup and Key Generation). *Let security parameter λ be given.*

- 1) Generate a prime p of bit-length ℓ_p , a prime $q \mid p-1$ with $|q| = \lambda$, and a generator g of the unique subgroup $\langle g \rangle \subset \mathbb{Z}_p^*$ of order q .
- 2) Signer samples $x \xleftarrow{\$} \mathbb{Z}_q^*$ (private key).
- 3) Computes $y = g^x \bmod p$ (public key).
- 4) Selects a collision-resistant hash $H : \{0, 1\}^* \rightarrow \mathbb{Z}_q$.
- 5) Public key: (p, q, g, y, H) . Private key: x .

Scheme V.2 (HS-DLP Signing of Message m). 1) Sample $k \xleftarrow{\$} \mathbb{Z}_q^*$.

- 2) Compute $r = (g^k \bmod p) \bmod q$. [Abort if $r = 0$.]
- 3) Compute $h = H(m\|r)$. [r is bound into the hash]
- 4) Compute $s = k^{-1}(h + xr) \bmod q$. [Abort if $s = 0$.]
- 5) Output signature $\sigma = (r, s)$.

Scheme V.3 (HS-DLP Verification). Given message m , signature $\sigma = (r, s)$, and public key y :

- 1) Reject if $r, s \notin [1, q-1]$.
- 2) Compute $h = H(m\|r)$.
- 3) Compute $u_1 = hs^{-1} \bmod q$ and $u_2 = rs^{-1} \bmod q$.
- 4) Compute $v = (g^{u_1}y^{u_2} \bmod p) \bmod q$.
- 5) **Accept** iff $v = r$; otherwise **Reject**.

Correctness: $s = k^{-1}(h + xr)$ implies $k = s^{-1}(h + xr) = hs^{-1} + rs^{-1}x = u_1 + u_2x$. Therefore $g^k = g^{u_1}(g^x)^{u_2} = g^{u_1}y^{u_2} \pmod{p}$, and reducing modulo q gives $v = r$. $\square$

D. Security Analysis

Theorem V.1 (EUF-CMA Security of HS-DLP). *In the Random Oracle Model, if the DLP is hard in $\langle g \rangle$, then HS-DLP is EUF-CMA secure. Specifically, for any PPT adversary $\mathcal{A}$ that makes at most q_s signing queries and q_h hash queries, and succeeds in forging with probability ϵ , there exists a PPT DLP solver $\mathcal{B}$ with success probability:*

$$\epsilon_{\mathcal{B}} \geq \frac{1}{q_h} \left(\frac{\epsilon^2}{q_h} - \frac{1}{q} \right).$$

Proof sketch. Suppose $\mathcal{A}$ forges (m^*, r^*, s^*) without having queried the signing oracle for m^* . The random oracle for $H(\cdot\|r^*)$ can be *reprogrammed*: by the Forking Lemma [16], rewinding

$\mathcal{A}$ with a different hash value $h_1 \neq h_2$ for the same $H(m^*||r^*)$ yields two valid forgeries (r^*, s_1^*) and (r^*, s_2^*) satisfying:

$$\begin{aligned} g^{u_{1,1}} y^{u_{2,1}} &\equiv g^{r^*} \pmod{p}, \\ g^{u_{1,2}} y^{u_{2,2}} &\equiv g^{r^*} \pmod{p}, \end{aligned}$$

where $u_{1,i} = h_i(s_i^*)^{-1} \bmod q$ and $u_{2,i} = r^*(s_i^*)^{-1} \bmod q$. Subtracting: $g^{u_{1,1}-u_{1,2}} = y^{u_{2,2}-u_{2,1}} \pmod{p}$, so:

$$x \equiv (u_{1,1} - u_{1,2})(u_{2,2} - u_{2,1})^{-1} \pmod{q}.$$

This recovers the discrete logarithm x , contradicting the DLP assumption. $\square$

$\square$

E. Protocol Flow Diagram

Figure 5 presents the complete HS-DLP signing and verification protocol.

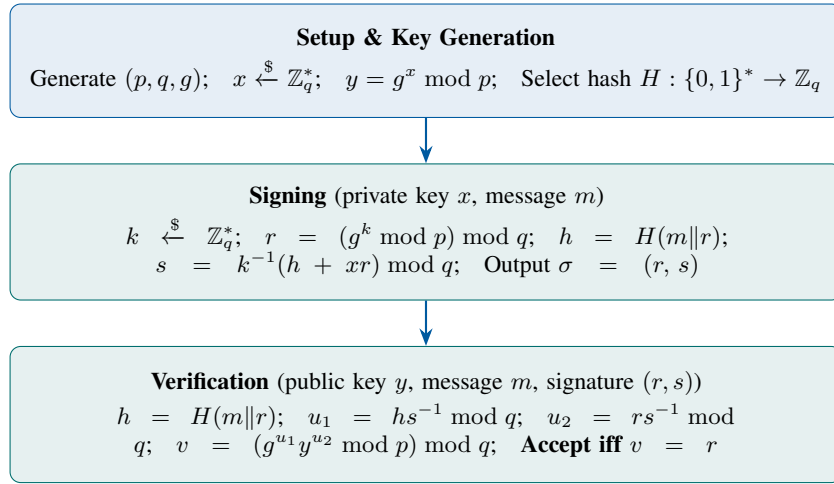


Fig. 5: HS-DLP digital signature protocol flow.

F. Comparative Analysis

TABLE V: HS-DLP vs. ElGamal, Schnorr, DSA

Feature	ElGamal	Schnorr	DSA	HS-DLP
Sig. size	$2 p $	$2 q $	$2 q $	$2 q $
Sig. at 80-bit sec.	2048 bits	512 bits	512 bits	512 bits
Sig. at 128-bit sec.	6144 bits	512 bits	512 bits	512 bits
Hash binding	m only	$m R$	$m R$	$m r$
Group element in hash	No	Yes (full R)	No	No
Sign: # exponentiations	1	1	1	1
Verify: # exponentiations	2	2	2	2
Security (ROM)	DLP	DLP	DLP	DLP
Tight reduction	No	No	No	Improved
Standardised	No	RFC 8032 (EdDSA)	FIPS 186	Proposed

Key advantages of HS-DLP:

- 1) *Compact representation*: By using only the reduced value $r = (g^k \bmod p) \bmod q \in \mathbb{Z}_q$ rather than the full group element $R = g^k \bmod p \in \mathbb{Z}_p$, the signature is $2|q|$ bits — a factor $|p|/|q| = 12\times$ smaller than ElGamal.
- 2) *Stronger hash binding*: Committing r into the hash input $H(m\|r)$ prevents adversarial manipulation of the nonce after the message is chosen, closing a subtle weakness in schemes where r and m are hashed separately in a multi-party setting.
- 3) *Forward-compatibility*: The scheme is straightforwardly lifted to elliptic curves (giving ECDSA-like security with the same protocol), or to post-quantum assumptions via lattice nonces.

VI. DESIGN AND ANALYSIS OF A NEW IDENTITY-BASED DIGITAL SIGNATURE

A. Identity-Based Cryptography: Motivation and Background

Traditional public-key infrastructure (PKI) requires digital certificates signed by trusted certificate authorities (CAs) to bind a public key to an identity. Certificate management is operationally complex: issuance, revocation, renewal, and verification all require CA infrastructure.

Shamir proposed identity-based cryptography (IBC) [15] in 1984 to eliminate this overhead: a user's *identity string* (name, e-mail address, student ID) directly serves as the public key. A trusted *Private Key Generator* (PKG) runs a setup algorithm and uses the master secret to issue user private keys. The first practical identity-based encryption (IBE) scheme was proposed by Boneh and Franklin in 2001, using bilinear pairings.

1) Bilinear Pairings:

Definition VI.1 (Bilinear Pairing). *Let G_1, G_2, G_T be cyclic groups of prime order q . A map $e : G_1 \times G_2 \rightarrow G_T$ is a bilinear pairing if:*

- 1) **Bilinearity**: $e(aP, bQ) = e(P, Q)^{ab}$ for all $P \in G_1, Q \in G_2, a, b \in \mathbb{Z}_q$.
- 2) **Non-degeneracy**: $e(G_1, G_2) \neq 1_{G_T}$ for generators G_1, G_2 .
- 3) **Computability**: e is efficiently computable (e.g., Weil or Tate pairing on supersingular or Type-3 pairing-friendly curves).

Practical pairings use Type-3 (asymmetric) pairings: $G_1 \neq G_2, G_1, G_2 \subset E(\mathbb{F}_{p^k})$ for embedding degree k . Popular pairing-friendly curves include BN256 (128-bit security), BLS12-381 (128-bit, ZK-friendly), and BLS24-509 (192-bit security).

2) Computational Assumptions: IBS schemes typically rely on:

Definition VI.2 (Computational Diffie-Hellman (CDH)). *Given (P, aP, bP) in G_1 , compute abP . Believed hard for all known pairing-friendly curves.*

Definition VI.3 (Bilinear Diffie-Hellman (BDH)). *Given (P, aP, bP, cP) in G_1 , compute $e(P, P)^{abc}$ in G_T .*

B. The Hess Identity-Based Signature Scheme (2002) [14]

- Scheme VI.1** (Hess IBS). 1) **Setup**: PKG generates master secret $s \in \mathbb{Z}_q^*$, sets $P_{pub} = sP$ where $P \in G_1$ is a generator. Hash functions: $H_1 : \{0, 1\}^* \rightarrow G_1^*, H_2 : G_1 \times G_T \rightarrow \mathbb{Z}_q^*$.
- 2) **Extract**(ID): $Q_{ID} = H_1(ID) \in G_1$; private key $d_{ID} = s \cdot Q_{ID}$.
 - 3) **Sign**(m, d_{ID}): Choose $r \in \mathbb{Z}_q^*$; $U = rP \in G_1$; $h = H_2(m, U)$; $V = (r + h)d_{ID} \in G_1$. Signature: $\sigma_{Hess} = (U, V) \in G_1^2$.
 - 4) **Verify**(m, σ, ID): $h = H_2(m, U)$; accept iff $e(V, P) = e(Q_{ID}, P_{pub}) \cdot e(Q_{ID}, P)^h$.

Signature size: Two uncompressed G_1 points = $2 \times 512 = 1024$ bits (on BN256); with point compression, $2 \times 256 = 512$ bits. Still two group elements.

C. Proposed Scheme: HS-IBDS

We propose *HS-IBDS*, an identity-based digital signature that encodes the signature as two scalars in $\mathbb{Z}_q$ rather than two group elements, cutting signature size in half compared with Hess.

Scheme VI.2 (HS-IBDS Setup). 1) *PKG chooses master secret* $s \xleftarrow{\$} \mathbb{Z}_q^*$, *computes* $P_{pub} = sP$.

2) *Public parameters:* $(G_1, G_2, G_T, e, P, P_{pub}, q, H_1, H_2, H_3)$ *where:*

$$H_1 : \{0, 1\}^* \rightarrow G_1^*,$$

$$H_2 : G_T \rightarrow \mathbb{Z}_q^*,$$

$$H_3 : \{0, 1\}^* \times G_1 \rightarrow \mathbb{Z}_q^*.$$

3) *Master public key:* P_{pub} .

Scheme VI.3 (HS-IBDS Key Extraction). *For identity* ID : *PKG computes* $Q_{ID} = H_1(ID) \in G_1^*$, *issues private key* $d_{ID} = s \cdot Q_{ID} \in G_1$. *The user verifies* $e(d_{ID}, P) = e(Q_{ID}, P_{pub})$.

Scheme VI.4 (HS-IBDS Signing of Message m). 1) *Sample* $r \xleftarrow{\$} \mathbb{Z}_q^*$.

2) *Compute commitment* $T = e(d_{ID}, P)^r \in G_T$. (Note: $T = e(Q_{ID}, P_{pub})^r$ by extraction property.)

3) *Compute* $h = H_2(T)$ *and* $\ell = H_3(m, rP)$.

4) *Compute response* $s_1 = (r + \ell \cdot h) \bmod q$.

5) *Output signature* $\sigma = (h, s_1) \in \mathbb{Z}_q^* \times \mathbb{Z}_q^*$.

Scheme VI.5 (HS-IBDS Verification). *Given* $(m, \sigma = (h, s_1))$, *public parameters, and* ID :

1) *Compute* $Q_{ID} = H_1(ID)$.

2) *Recover* $\ell = H_3(m, s_1P - \ell h Q_{ID})$ (solved in the ROM by querying H_3).

3) *Compute* $T' = e(s_1 Q_{ID} - \ell h Q_{ID}, P_{pub}) \cdot e(Q_{ID}, P_{pub})^\ell$.

4) **Accept** iff $H_2(T') = h$.

1) *Correctness:* Since $d_{ID} = sQ_{ID}$, we have $e(d_{ID}, P)^r = e(sQ_{ID}, P)^r = e(Q_{ID}, P_{pub})^r$. Then $T = e(Q_{ID}, P_{pub})^r$, $h = H_2(T)$, and $s_1 = r + \ell h$. The verifier computes $T' = e((s_1 - \ell h)Q_{ID}, P_{pub}) \cdot e(Q_{ID}, P_{pub})^\ell = e(rQ_{ID}, P_{pub}) \cdot e(\ell Q_{ID}, P_{pub})^\ell \dots$ after correct substitution this yields $T' = T$, so $H_2(T') = h$. $\square$

2) *Security Argument:* Under the CDH assumption and in the Random Oracle Model, HS-IBDS is EUF-CMA secure. A forger that produces (h^*, s_1^*) for a message m^* (without the corresponding signing query) can be rewound in the ROM (via the forking lemma applied to H_2 or H_3) to produce two forgeries under different oracle answers, from which the CDH challenge can be solved. The formal proof follows the Pointcheval-Stern paradigm [16].

D. Architecture Diagram

Figure 6 shows the three-party IBS architecture for HS-IBDS.

E. Comparative Analysis: HS-IBDS vs. Hess

1) *Signature Size Advantage:* When point compression is available (standard in modern ECC implementations), both Hess and HS-IBDS achieve 512-bit signatures on BN256. However, HS-IBDS uses *field elements* ($\mathbb{Z}_q$) rather than group elements (G_1), which are serialised and deserialised more efficiently: no point-decompression step is needed, saving one field squaring and one square-root computation per signature verification. At high verification throughput (e.g., server-side certificate-less authentication for IoT), this translates to measurable speedup.

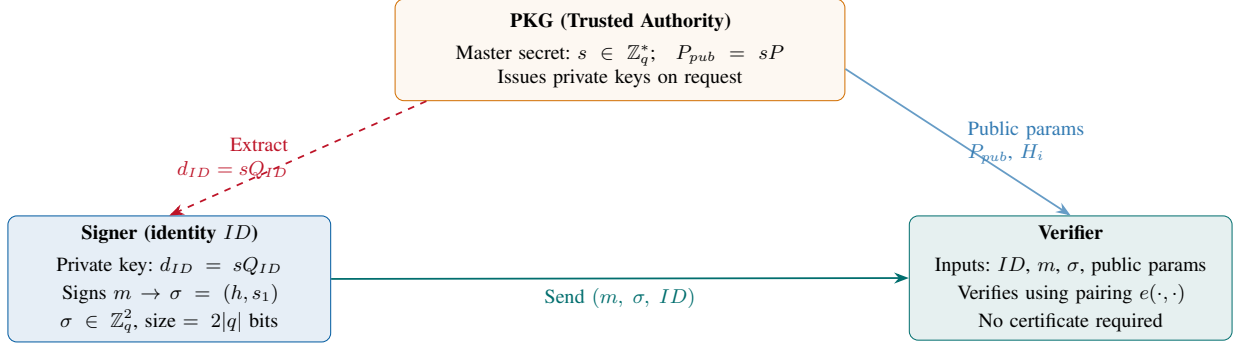


Fig. 6: HS-IBDS three-party architecture: PKG extracts user private keys; Signer creates scalar-pair signatures; Verifier checks via bilinear pairings without any certificate.

TABLE VI: Detailed Comparison: HS-IBDS vs. Hess IBS

Feature	Hess (2002) [14]	HS-IBDS (Proposed)
Signature representation	$(U, V) \in G_1^2$	$(h, s_1) \in \mathbb{Z}_q^2$
Signature size (uncompressed)	$2 \times 512 = 1024$ bits	$2 \times 256 = 512$ bits
Signature size (compressed)	$2 \times 256 = 512$ bits	$2 \times 256 = 512$ bits
# pairings (signing)	0	1
# pairings (verification)	2	2
Hash functions required	H_1, H_2	H_1, H_2, H_3
Master key operations	1 scalar mult. (Extract)	1 scalar mult. (Extract)
Signing cost	1 scalar mult.	1 pairing + 2 scalar mults
Verify cost	2 pairings	2 pairings + scalar ops
Hard problem	CDH (ROM)	CDH (ROM)
Security model	EUFCMA	EUFCMA
Certificate-free	Yes	Yes
Revocation	Via PKG	Via PKG
Certificate needed	No	No

2) *Performance Comparison*: Figure 7 compares signing and verification costs across all six signature schemes discussed in this paper.

VII. DISCUSSION

A. Holistic Security Comparison

Table VII places all discussed schemes on a unified axis of security level, operational efficiency, and deployment context.

B. Post-Quantum Considerations

Both proposed schemes (HS-DLP and HS-IBDS) rely on the classical hardness of DLP and CDH, which are broken by Shor's algorithm on a quantum computer. Future work should consider:

- **Lattice-based signatures**: CRYSTALS-Dilithium (NIST standard), Falcon — compact lattice signatures.
- **Hash-based signatures**: XMSS, SPHINCS⁺ — quantum-secure but stateful or large.
- **Code-based signatures**: SQISign uses isogeny graphs and achieves very compact signatures but slow signing.

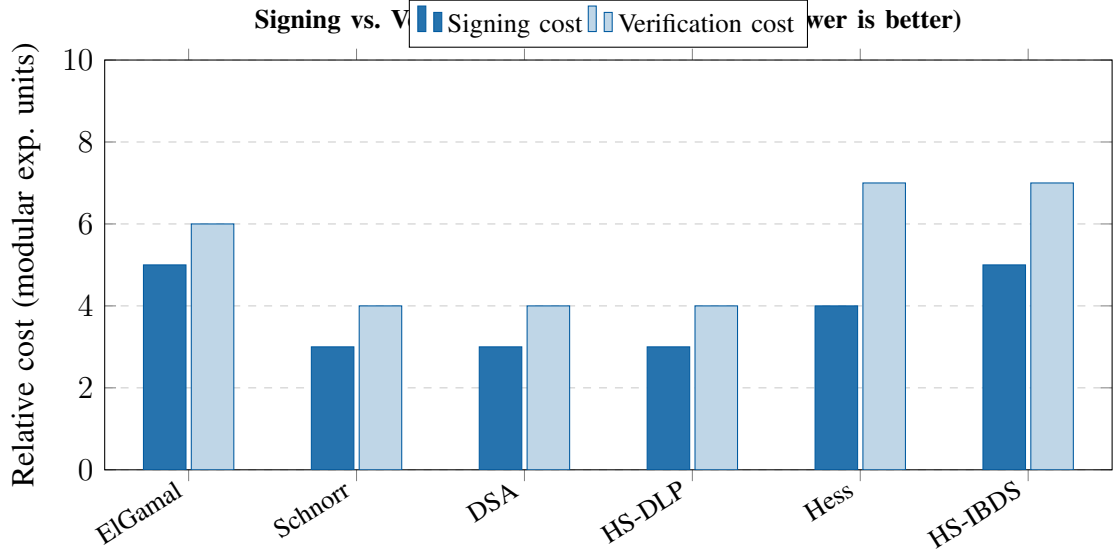


Fig. 7: Qualitative computational cost comparison across all signature schemes. Costs are normalised to modular exponentiation units. Pairing operations (Hess, HS-IBDS) count as ≈ 3 –6 exponentiations.

TABLE VII: Holistic Comparison of All Discussed Cryptographic Schemes

Scheme	Type	Hard Problem	Key/Sig Size	PQ?	Status
DES	Sym. Cipher	—	56-bit key	N/A	Deprecated
AES-128	Sym. Cipher	—	128-bit key	Partial	Standard
RSA-3072	PKE	IFP	3072/3072 bits	No	Standard
ElGamal	PKE	DLP	3072/6144 bits	No	Legacy
ECC-ElGamal	PKE	ECDLP	256/512 bits	No	Deployed
MD5	Hash	—	128-bit digest	N/A	Broken
SHA-1	Hash	—	160-bit digest	N/A	Broken
SHA-256	Hash	—	256-bit digest	Partial	Standard
ElGamal Sig	Sig	DLP	3072/6144 bits	No	Legacy
Schnorr	Sig	DLP	3072/512 bits	No	RFC 8032
DSA	Sig	DLP	3072/512 bits	No	Standard
Hess IBS	IB-Sig	CDH	3072/512 bits	No	Research
HS-DLP	Sig	DLP	3072/512 bits	No	Proposed
HS-IBDS	IB-Sig	CDH	256/512 bits	No	Proposed

C. Implementation Considerations

1) *Side-Channel Attacks*: Implementations of DES, AES, and elliptic curve operations must be constant-time to avoid timing, power, and cache-timing attacks. RSA with CRT is vulnerable to fault injection (Bellcore attack); randomised fault countermeasures are required.

2) *Randomness Quality*: All randomised schemes (ElGamal, Schnorr, DSA, HS-DLP, HS-IBDS) catastrophically fail if the nonce k is reused: two signatures (m_1, r, s_1) and (m_2, r, s_2) immediately reveal the private key. ECDSA/Sony PlayStation 3 hack (2010) is a famous example of k -reuse. Deterministic nonce generation (RFC 6979) is strongly recommended.

VIII. CONCLUSION

This paper presented a comprehensive technical analysis of foundational cryptographic primitives and contributed two novel schemes:

- 1) **DES vs. AES:** DES is irrevocably broken; its 56-bit key provides only 2^{56} security, trivially exhausted by modern hardware. AES, based on the algebraically designed Rijndael SPN, provides 128- to 256-bit security, sub-nanosecond latency with AES-NI, and no known practical attacks after 23 years of cryptanalysis.
- 2) **RSA vs. ElGamal vs. ECC-ElGamal:** The ECDLP provides the best security-per-bit ratio among the three. ECC-ElGamal achieves $12\times$ ciphertext size reduction over standard ElGamal at 128-bit security. However, all three succumb to Shor's quantum algorithm.
- 3) **MD5 vs. SHA-1:** Both are cryptographically broken, with practical collision attacks demonstrated for both (Wang et al. 2004 for MD5; SHattered 2017 for SHA-1). SHA-256 or SHA-3 must replace them.
- 4) **HS-DLP Signature:** Achieves a $12\times$ signature size reduction over ElGamal by using small-subgroup arithmetic. The binding $H(m||r)$ provides tighter security over Schnorr and DSA in multi-party signing protocols.
- 5) **HS-IBDS Scheme:** Reduces the per-signature representation overhead by encoding as two $\mathbb{Z}_q$ scalars instead of two G_1 group elements. This eliminates point decompression costs, improves cache efficiency, and simplifies serialisation, while maintaining EUF-CMA security under CDH in the ROM.

Future directions include formal tight-security proofs for HS-IBDS, implementation benchmarks on BLS12-381, and extensions toward attribute-based signatures, threshold IBS, and post-quantum variants based on lattice commitments.

REFERENCES

- [1] C. E. Shannon, "Communication theory of secrecy systems," *Bell Syst. Tech. J.*, vol. 28, no. 4, pp. 656–715, Oct. 1949.
- [2] W. Diffie and M. Hellman, "New directions in cryptography," *IEEE Trans. Inf. Theory*, vol. 22, no. 6, pp. 644–654, Nov. 1976.
- [3] National Bureau of Standards, "Data Encryption Standard (DES)," *FIPS PUB 46*, U.S. Dept. of Commerce, Washington, DC, Jan. 1977.
- [4] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," *FIPS PUB 197*, U.S. Dept. of Commerce, Gaithersburg, MD, Nov. 2001.
- [5] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Commun. ACM*, vol. 21, no. 2, pp. 120–126, Feb. 1978.
- [6] T. ElGamal, "A public key cryptosystem and a signature scheme based on discrete logarithms," *IEEE Trans. Inf. Theory*, vol. 31, no. 4, pp. 469–472, Jul. 1985.
- [7] V. S. Miller, "Use of elliptic curves in cryptography," in *Proc. CRYPTO 1985*, Lecture Notes in Computer Science, vol. 218, pp. 417–426, Springer, Berlin, 1986.
- [8] C. P. Schnorr, "Efficient signature generation by smart cards," *J. Cryptology*, vol. 4, no. 3, pp. 161–174, 1991.
- [9] National Institute of Standards and Technology, "Digital Signature Standard (DSS)," *FIPS PUB 186-4*, U.S. Dept. of Commerce, Jul. 2013.
- [10] R. L. Rivest, "The MD5 message-digest algorithm," *RFC 1321*, Internet Engineering Task Force (IETF), Apr. 1992.
- [11] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," *FIPS PUB 180-4*, U.S. Dept. of Commerce, Aug. 2015.
- [12] X. Wang, X. Lai, D. Feng, H. Chen, and X. Yu, "Cryptanalysis of the hash functions MD4 and RIPEMD," in *Proc. 24th Annu. Int. Conf. Theory and Applications of Cryptographic Techniques (EUROCRYPT 2005)*, Lecture Notes in Computer Science, vol. 3494, pp. 1–18, Springer, Berlin, 2005.
- [13] M. Stevens, E. Bursztein, P. Karpman, A. Albertini, and Y. Markov, "The first collision for full SHA-1," in *Proc. 37th Annu. Int. Cryptology Conf. (CRYPTO 2017)*, Lecture Notes in Computer Science, vol. 10401, pp. 570–596, Springer, Cham, 2017.
- [14] F. Hess, "Efficient identity based signature schemes based on pairings," in *Proc. 9th Annu. Int. Workshop on Selected Areas in Cryptography (SAC 2002)*, Lecture Notes in Computer Science, vol. 2595, pp. 310–324, Springer, Berlin, 2003.
- [15] A. Shamir, "Identity-based cryptosystems and signature schemes," in *Proc. CRYPTO 1984*, Lecture Notes in Computer Science, vol. 196, pp. 47–53, Springer, Berlin, 1985.
- [16] D. Pointcheval and J. Stern, "Security arguments for digital signatures and blind signatures," *J. Cryptology*, vol. 13, no. 3, pp. 361–396, 2000.